SESSION DE RATTRAPAGE – JEE1 ET JEE2

Application CRUD de facturation

TP1 : Spring Boot + HTML / CSS + MySQL

Clavstel El Schaddai BOYAOU

Classe : M1 GELO

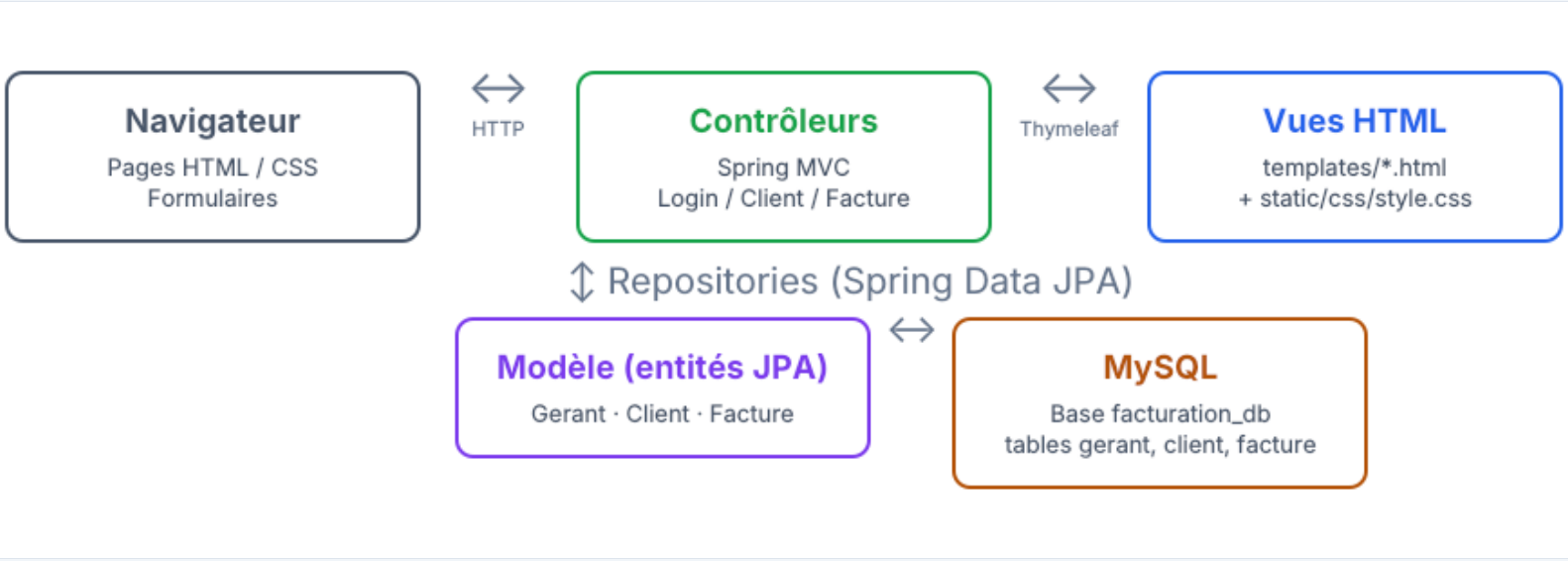
Plan

- Contexte et objectifs du TP1
- Architecture MVC et technologies
- Modèle de données MySQL
- Sécurité : authentification du gérant
- Routes de l'application
- Présentation de la plateforme (interfaces)
- Conclusion et perspectives (vers le TP2)

Contexte et objectifs

- **Besoin :** gérer les clients et les factures d'une entreprise
- **Acteur :** le gérant, seul utilisateur du système
- **Fonctionnalités :** authentification, puis ajout, modification, suppression et recherche des clients et des factures
- **Technologies imposées :** Spring Boot, HTML, CSS, MySQL

Architecture MVC



- Le contrôleur reçoit la requête, interroge le modèle via les repositories, puis choisit la vue HTML
- Les pages sont générées côté serveur par Thymeleaf puis envoyées au navigateur

Technologies utilisées

- **Backend :** Java 17, Spring Boot 3.2, Spring MVC, Spring Data JPA (Hibernate)
- **Vues :** HTML5 + Thymeleaf, CSS3 (feuille style.css)
- **Base de données :** MySQL 8 (base facturation_db)
- **Sécurité :** Spring Security (formulaire de connexion) + BCrypt
- **Validation :** Bean Validation (jakarta.validation)
- **Outils :** Maven

Modèle de données MySQL



Facture(Idfact, montant, objet, IdClient) – Client(Idclient, nom, tel) – Gérant(Id, nom, login, password)

Sécurité : authentification du gérant

- Page **/login** : formulaire HTML (login + mot de passe)
- Spring Security compare le mot de passe saisi au hash **BCrypt** stocké dans la table gerant
- En cas de succès : session ouverte, redirection vers /clients
- Toutes les autres URL exigent le rôle **GERANT** : sinon redirection vers /login
- Protection **CSRF** : chaque formulaire POST contient un jeton généré par Thymeleaf
- Déconnexion : bouton en haut à droite (POST /logout)

Routes de l'application

Méthode	URL	Action
GET	/login	Formulaire de connexion
GET	/clients?q=	Lister / rechercher les clients
GET · POST	/clients/nouveau · /clients/save	Afficher le formulaire · enregistrer (ajout / modification)
GET	/clients/{id}/modifier	Modifier un client
POST	/clients/{id}/supprimer	Supprimer un client (et ses factures)
GET	/factures?q=	Lister / rechercher les factures
GET · POST	/factures/nouveau · /factures/save	Formulaire · enregistrer
GET · POST	/factures/{id}/modifier · /supprimer	Modifier · supprimer une facture

Plateforme : connexion du gérant

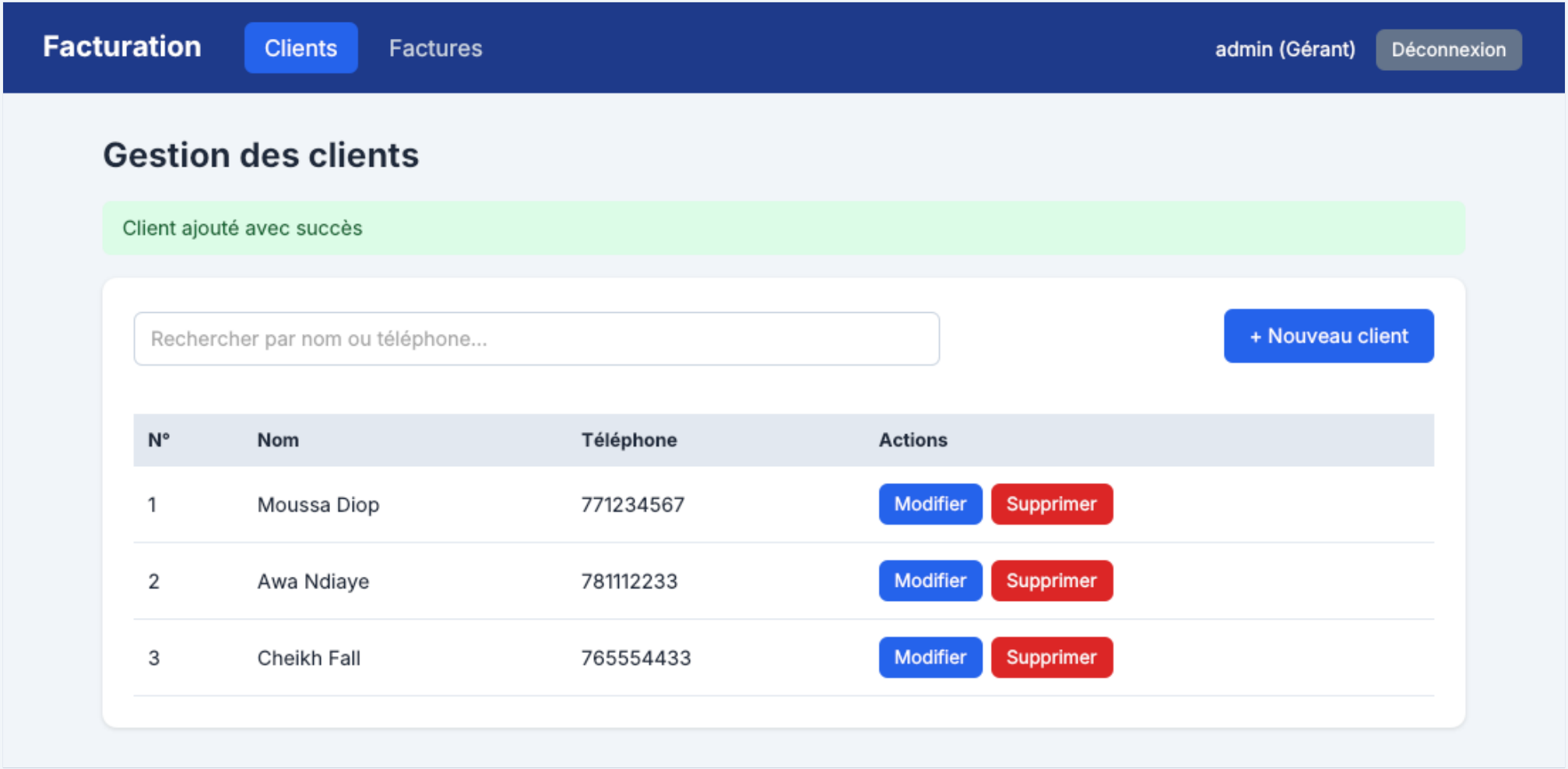
- Accès réservé au gérant ; message d'erreur si identifiants incorrects



Page de connexion

Gestion des clients

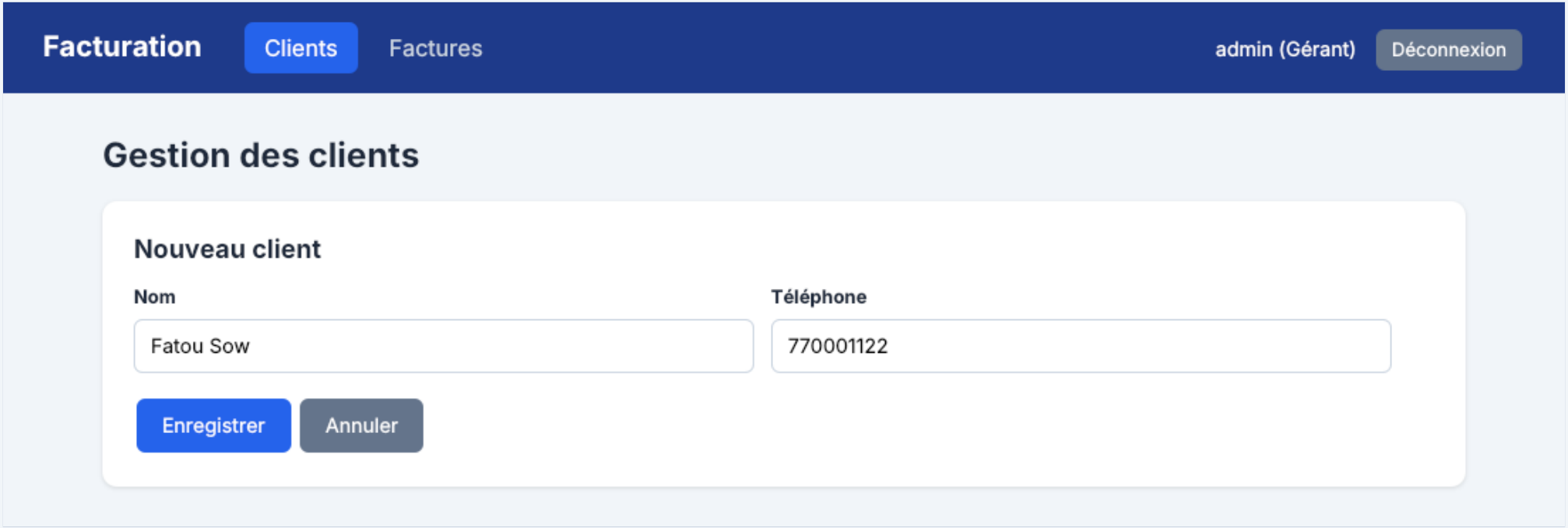
- Ajouter, modifier, supprimer, rechercher (nom ou téléphone) ; message de confirmation en vert



Liste des clients, recherche et actions

Ajout et modification d'un client

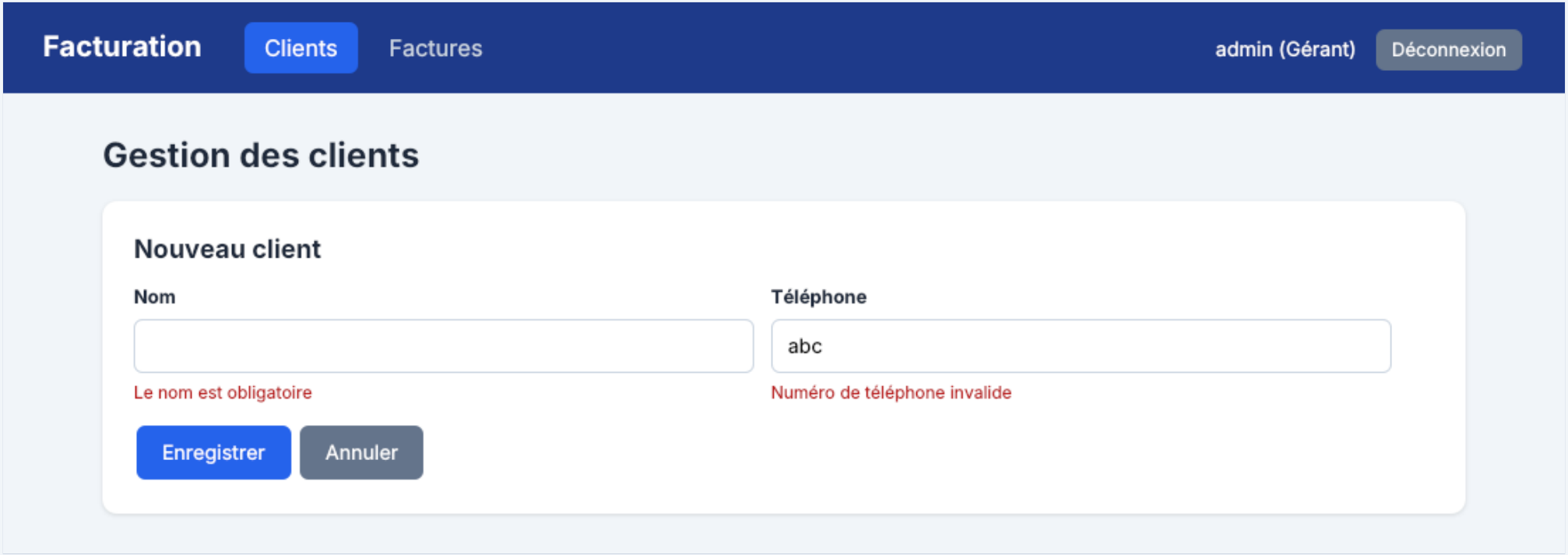
- Le même formulaire sert à l'ajout et à la modification



Formulaire client

Validation des formulaires

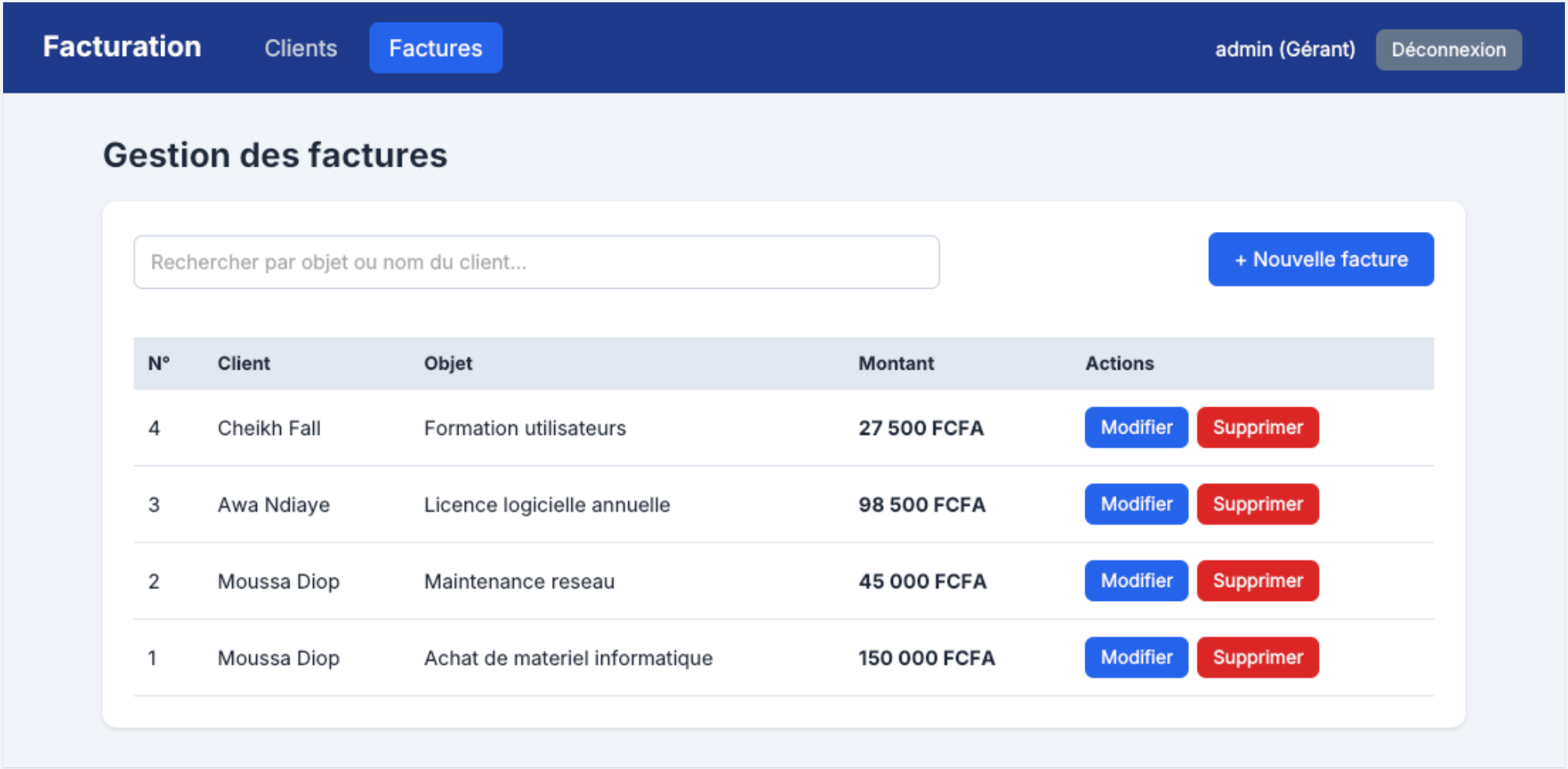
- Validation côté serveur : nom obligatoire, téléphone au bon format



Messages d'erreur sous les champs

Gestion des factures

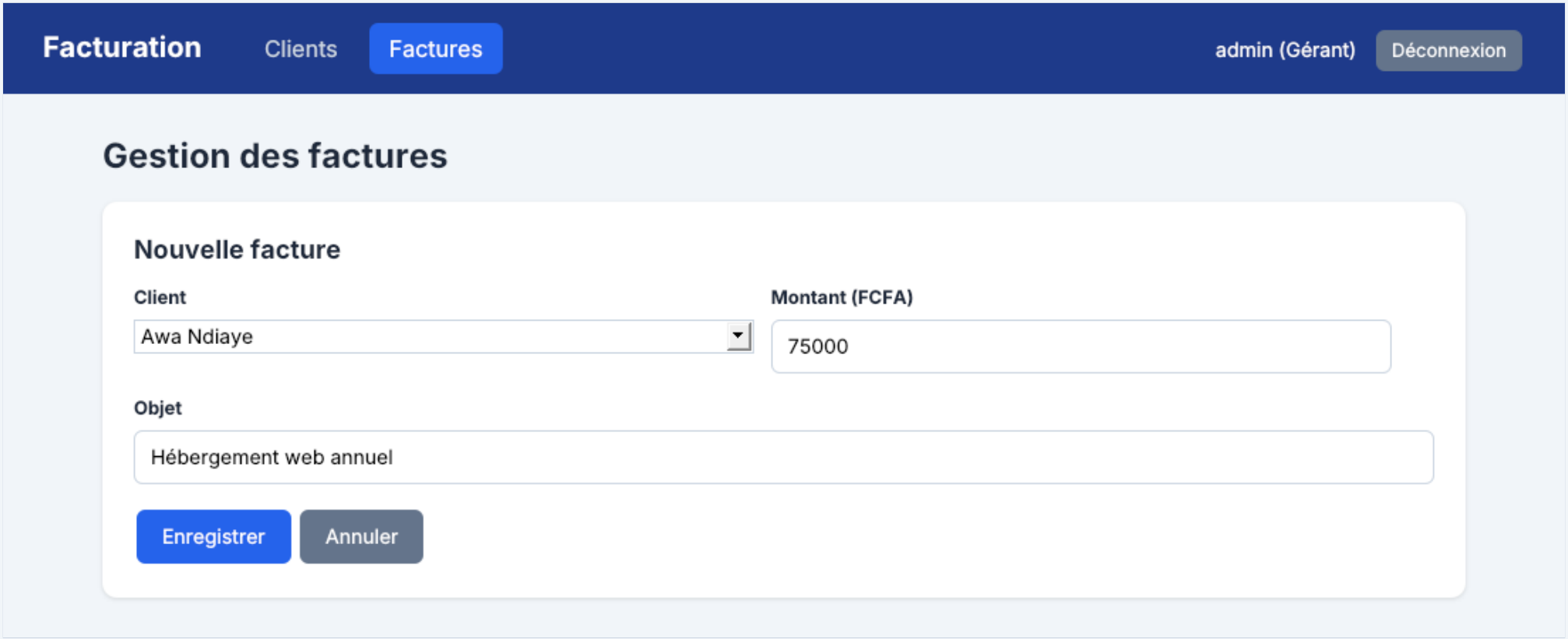
- Recherche par objet ou par nom du client ; montants formatés en FCFA



Liste des factures, recherche et actions

Ajout d'une facture

- Le client est choisi dans une liste déroulante (clé étrangère IdClient)



Formulaire facture : client, montant, objet

Conclusion et perspectives

- Application conforme au sujet du TP1 : Spring Boot + HTML/CSS + MySQL
- Authentification du gérant, CRUD et recherche complets sur clients et factures
- Sécurité : BCrypt, CSRF, accès réservé au rôle GERANT, suppression en cascade
- **Vers le TP2 :** même architecture, frontend React, base MongoDB, accès client

Merci de votre attention